



USERNAME ENUMERATION VIA ACCOUNT LOCK



ACCOUNT LOCKED!



ATTEMPTS



HAROON SHAHID

Lab: Username Enumeration via Account Lock

This lab is vulnerable to username enumeration. It uses account locking, but this contains a logic flaw. To solve the lab, enumerate a valid username, brute-force this user's password, then access their account page.

- Candidate usernames
 - Candidate passwords
-

Introduction

This lab is vulnerable to **username enumeration** because of a logic flaw in the application's account-locking mechanism.

The application uses account locking as a **brute-force protection** mechanism. The idea is that if someone keeps entering the wrong password for an account, the application temporarily locks that account so the attacker cannot continue guessing passwords.

However, the way this application handles the lock creates an information leak. By observing how the application responds to different usernames, we can determine which usernames actually exist.

The goal of this lab is:

1. Find a valid username from the candidate usernames list.
2. Find that user's password from the candidate passwords list.
3. Wait for the account lock to expire if necessary.
4. Log in using the valid username and correct password.
5. Access the user's account page.

Understanding the login behavior

Let's first understand what happens when we submit different types of credentials.

Invalid username + any password

If we enter a username that does not exist, the application responds with:

Invalid username or password

If we continue sending requests using that same invalid username, we continue getting the same type of response.

For example:

Invalid username + wrong password → Invalid username or password

Invalid username + wrong password → Invalid username or password

Invalid username + wrong password → Invalid username or password

...and so on

The application does not eventually give us an account-lock response for that username.

This makes sense because the username does not belong to an existing account. There is no real account associated with that username for the application to lock.

Valid username + invalid password

Now consider what happens when we use a **valid username** but provide the wrong password.

The first few requests produce the normal failed-login response:

Valid username + wrong password → Invalid username or password

But after **3 failed attempts**, the next attempt triggers the account-lock protection.

So the behavior is approximately:

Attempt 1 → Invalid username or password

Attempt 2 → Invalid username or password

Attempt 3 → Invalid username or password

Attempt 4 → Account lock

The application then tells us something like:

You have made too many incorrect login attempts. Try again after 1 minute(s).

The account is locked for **1 minute**.

This means that even if we know the correct password while the account is locked, we cannot immediately use it to log in. But we won't get any type of message also and remember this point because it will help us in finding the password.

An important difference between valid and invalid usernames

Now we have two different behaviors:

Invalid username

Attempt 1 → Invalid username or password

Attempt 2 → Invalid username or password

Attempt 3 → Invalid username or password

Attempt 4 → Invalid username or password

Attempt 5 → Invalid username or password

...

Valid username

Attempt 1 → Invalid username or password

Attempt 2 → Invalid username or password

Attempt 3 → Invalid username or password

Attempt 4 → You have made too many incorrect login attempts. Try again after 1 minute(s).

This difference is the key to the entire lab.

If a username eventually causes the account-lock response, we have evidence that the username belongs to an existing account.

Therefore, the application's account-lock mechanism unintentionally allows us to **enumerate valid usernames**.

But how do we know that the account is actually locked?

If we already knew the lab's behavior, we could simply say:

"After 3 failed attempts, the account is locked for 1 minute."

But when solving the lab from scratch, we should pretend that we don't already know the exact threshold.

We only know that the application has some kind of account-locking protection.

Therefore, we need to test the candidate usernames multiple times and observe whether the application's response changes.

For example, if we test a username several times and continue receiving:

Invalid username or password

we don't yet know whether that username is invalid or whether we simply haven't reached the lock threshold.

So we need to send enough requests to each candidate username to give the account-lock mechanism a chance to trigger.

In this lab, PortSwigger's solution uses **5 requests per username**.

The important idea is not that "5" is a magic number.

The idea is:

We repeat each candidate username several times so that, if it is valid, we have enough attempts to trigger the account-lock behavior.

When solving a real authorized application, we would determine the behavior carefully rather than blindly assuming a particular number of attempts.

What happens while the account is locked?

Once a valid account has been locked, the lock remains active for approximately 1 minute.

During this period, submitting the correct credentials does not immediately log us in.

Interestingly, the response does not necessarily continue to show:

You have made too many incorrect login attempts.

when we submit the correct credentials.

Instead, in this lab, the login does not succeed while the lock is active, and the response can appear like a normal login form without the usual error message.

The important thing is:

The account remains locked until the lock period expires.

So even if we discover the correct password, we may need to wait before we can successfully log in.

Can we reset the lock using an invalid username?

This is another important observation.

In the previous authentication lab, we learned that sometimes another type of login attempt can interfere with or reset brute-force protection.

We need to determine whether that works here.

Suppose we do this:

Valid username + wrong password → Failed

Valid username + wrong password → Failed

Valid username + wrong password → Failed

Valid username + wrong password → Account locked

Now, while the account is locked, we submit:

Invalid username + anything → Invalid username or password

Then we immediately try the previously locked valid username again:

Valid username + wrong password → The account is still locked.

Therefore, submitting an invalid username does **not** reset the lock timer.

This is important because it means we cannot bypass this protection by simply switching between valid and invalid usernames.

The lock is associated with the valid account and continues until the lock period expires.

Why does the lock only reveal information for valid usernames?

This is the main logical problem with the application's implementation.

An account lock is supposed to protect an **existing account**.

If the username does not exist:

Username: **abcxyz**

there is no actual account called **abcxyz** to lock.

Therefore, the application continues returning the normal **Invalid username or password** response.

But when the username does exist:

Username: **existing-user**

the application can track failed attempts against that account and eventually lock it.

So we get:

Non-existing username → Normal error response → No account lock

versus:

Existing username → Failed password attempts → Account lock → Different response

That difference reveals whether the username exists.

This is exactly what **username enumeration** means in this lab.

Finding the valid username

We are given a **candidate usernames list**.

Our goal is to test these usernames and find the one that behaves differently.

We don't initially know which username is valid.

Therefore, we send multiple login requests for each candidate username using an incorrect password.

Conceptually:

Username 1 → several attempts

Username 2 → several attempts

Username 3 → several attempts

Username 4 → several attempts

...

Most usernames will continue returning:

Invalid username or password

But the valid username will eventually trigger:

You have made too many incorrect login attempts.

In Caido, we can compare the responses and look for the username whose response is different.

In this particular lab, the account-lock response is also **longer** than the normal invalid-login response because it contains a different error message.

So response length is useful for quickly spotting the unusual response, and then we can inspect the response body to confirm the different error message.

Once we find that username, we have successfully enumerated a valid username.

Finding the password

Now that we have a valid username, the next step is to find its password.

We are given a **candidate passwords list**.

We keep the username fixed and test the candidate passwords one by one.

For incorrect passwords, we normally receive an error such as:

Invalid username or password

If we make enough incorrect attempts, we can also encounter:

You have made too many incorrect login attempts.

The correct password behaves differently.

When the correct username & password is submitted and the account is locked, you won't get any type of message which can decrease the length of the response. And help us identify that response by length.

However, response length is only a useful indicator for this particular lab. It is not a universal rule that a correct password will always produce a shorter response.

What if the account becomes locked while testing passwords?

This is important.

While testing the candidate passwords, we may trigger the account-lock mechanism because many of the candidate passwords will be incorrect.

If that happens, we cannot immediately log in even if we have already identified the correct password.

We need to wait for the **1-minute lock period** to expire.

We won't see any message in case of correct credentials but we won't be able to log in until the lock period expires. So in case of correct credentials nothing will happen, you will stay on the same login page.

Once the lock has reset, we can submit:

Valid username + Correct password

and successfully log in.

Complete attack flow

The entire attack can therefore be understood as:

Candidate usernames



Test each username multiple times



Observe response differences



Look for account-lock response



Identify valid username



Use candidate passwords



Test passwords against valid username



Compare responses



Identify response with no login error



Correct password found



Wait for account lock to expire if necessary → Log in → Access the user's account page → Lab solved

The actual vulnerability

The important thing to understand is that the vulnerability is **not simply the existence of account locking**.

Account locking is supposed to be a security feature.

The problem is that the application gives us a different observable response depending on whether the username belongs to an existing account.

Because an attacker can observe this difference, the application's brute-force protection becomes an **information leak**.

That information leak allows us to determine which usernames are valid.

Therefore, the lab is a **username enumeration vulnerability caused by account-locking behavior**.

Summary

If you enter invalid username, the application responds with Invalid username or password message. No matter how many attempts you try/the requests you send. I mean you will always get Invalid username or password as long as the username is invalid regardless of the number of tries/attempts and not that you get an account lock message for 1 minute.

But if you enter a valid username and invalid password, after 3 failed attempts on 4th attempt, access to that account is locked for 1 minute, even if you enter valid username and password for that account after that. But if you enter a valid username and password when the account lock timer is running, you won't be able to access the account but you won't get the account lock timer message. And not Invalid username or password message also. In fact, you won't get any error or account lock message, nothing just a normal login form and when the timer is not completed and you enter the correct credentials and click Log in, nothing happens until the timer resets.

But how do I know that after 3 failed attempts, access to the account is blocked for 1 minute? Well, I've already solved the lab, but here I will solve the lab as if I don't know this.

We are given a Candidate usernames list and we have to find a valid user and to do that we have to send requests with every user a minimum of 4 times. I mean 4 requests with every user. But at first we don't know this exact number 4 because we don't know the valid username. So we are gonna send 15 to 20 requests for every user because I told you this number but if I had not told you then what would you have done? Obviously send maybe 15-20 or even more than these requests in real web apps because you don't know when you are gonna get that error if it exists.

And if the timer is running and you enter invalid username, you will still get Invalid username or password message but it won't reset the timer like we did this kind of method in the previous lab but that won't work here.

So if you try a valid username and invalid password 4 times and got an account lock for 1 minute message and then try an invalid username and got Invalid username or password message and then again try to enter that same valid username and invalid password in that same minute, you will still get an account lock for 1 minute message. So the point is that entering an invalid username after an account lock doesn't reset the timer.

But there is one question and that is like why get this lock thing only for valid users and the reason is that if the user does not exist, how is the application gonna lock an account with that username because the username does not exist at the first. Also if someone enters invalid user, the developer might wonder what is the point of displaying an account lock message if the attacker does not even know the username, let alone the password.

So if the account is locked after 3 attempts, that means that user is a valid user but we will try 15-20 requests per user because I want you to assume we don't know this number 3.

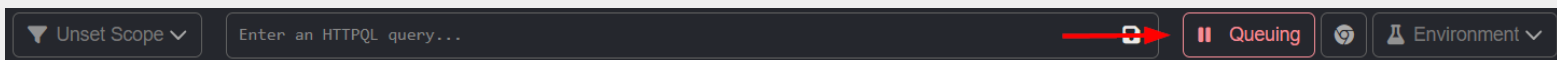
And to find the password, if we get no Invalid username or password message or account lock for 1 minute message, then that's the correct password and the way we are gonna identify that is that the length of this response will be a little smaller than the request where you get some message.

Now let's get started.

Solution

Step 1:

Go to My Account and turn Queuing on in Caido.



Step 2:

Enter any dummy data in the Login form and hit **Log in**. The request will be intercepted in Caido. Send it to **Automate**.

Choose what to intercept						
☒ Requests ☐ Responses ☐ Websockets						
ID	Host	Method	Path	Query	Exten...	Sent At
3	0a4e006303fd2...	POST	/login			2026-09-17 03:32:04

Step 3:

Go to Automate and select the attack type as Matrix.

Step 4:

Now mark the username value as placeholder and select Simple list as payload type. Copy the candidate username list and paste it.

The screenshot shows the Burp Suite Matrix tool interface. The 'Request' tab displays an HTTP POST request to `/login` on `https://0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net`. The 'Payload' tab is active, showing a 'Simple List' payload type with a list of 20 usernames. The 'Placeholder' field is set to 'test'. A blue arrow points to the 'test' placeholder in the request body.

```
1 POST /login HTTP/1.1
2 Host: 0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net
3 Connection: keep-alive
4 Content-Length: 27
5 Cache-Control: max-age=0
6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
7 sec-ch-ua-mobile: ?0
8 sec-ch-ua-platform: "Windows"
9 Upgrade-Insecure-Requests: 1
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
12 Origin: https://0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: https://0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net/login
19 Accept-Encoding: gzip, deflate, br, zstd
20 Accept-Language: en-US,en;q=0.9
21 Cookie: session=h0dt7gytPkdLwT60zIAHkEpwI3Fux9HQ
22
23 username=test&password=test
```

Step 5:

We are gonna send 5 requests for every user so for that, mark the password value as placeholder and select Simple list as payload type and in the list write random 5 entries. This will test every entry against every single username, sending a single username in 5 requests.

The screenshot shows the Burp Suite Matrix tool interface. The 'Request' tab displays an HTTP POST request to `/login` on `https://0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net`. The 'Payload' tab is active, showing a 'Simple List' payload type with a list of 5 'test' entries. The 'Placeholder' field is set to 'test'. A blue arrow points to the 'test' placeholder in the request body.

```
1 POST /login HTTP/1.1
2 Host: 0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net
3 Connection: keep-alive
4 Content-Length: 27
5 Cache-Control: max-age=0
6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
7 sec-ch-ua-mobile: ?0
8 sec-ch-ua-platform: "Windows"
9 Upgrade-Insecure-Requests: 1
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
12 Origin: https://0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: https://0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net/login
19 Accept-Encoding: gzip, deflate, br, zstd
20 Accept-Language: en-US,en;q=0.9
21 Cookie: session=h0dt7gytPkdLwT60zIAHkEpwI3Fux9HQ
22
23 username=test&password=test
```

Hit Run.

Enter an HTTPQL query...

ID	Payload 2	Payload 1	Status	Length	Round-trip Time (ms)
354	test	app	200	3417	672
355	test	app	200	3417	747
1	test	carlos	200	3365	772
2	test	carlos	200	3365	785
3	test	carlos	200	3365	822
4	test	carlos	200	3365	771

505 requests Reset preference

Request Pretty Raw Response Pretty Raw Preview

1 POST /login HTTP/1.1
2 Host: 0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net
3 Connection: close
4 Content-Length: 26
5 Cache-Control: max-age=0
6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
7 sec-ch-ua-mobile: ?0
8 sec-ch-ua-platform: "Windows"
9 Upgrade-Insecure-Requests: 1
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
12 Origin: https://0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document

Web Security Academy Username enumeration via account lock LAB Not solved
Back to lab description >
Home | My account
Login
You have made too many incorrect login attempts. Please try again in 1 minute(s)
Username
Password
Log in
3417 bytes | 747ms

We can see that for **app** we got length greater than the rest because the response has a bigger message which increases the response length.

Step 6:

To find the password, fix the username value as app and select the attack type as All and highlight the password field as placeholder. Copy the Candidate passwords list and select the Payload type as Simple list, paste in the passwords and hit Run.

https://0a7e00400433709e8 All Run Select enabled rules

Request Clear - + Payload Preprocessors Settings

1 POST /login HTTP/1.1
2 Host: 0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net
3 Connection: keep-alive
4 Content-Length: 27
5 Cache-Control: max-age=0
6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
7 sec-ch-ua-mobile: ?0
8 sec-ch-ua-platform: "Windows"
9 Upgrade-Insecure-Requests: 1
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
12 Origin: https://0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: https://0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net/login
19 Accept-Encoding: gzip, deflate, br, zstd
20 Accept-Language: en-US,en;q=0.9
21 Cookie: session=h0dt7gytPkdLwT60z1AHkEpwI3Fux9HQ
22
23 username=app&password=test

Payload # Placeholder
1 test

Type
Simple List

- 81 ashley
- 82 nicole
- 83 chelsea
- 84 biteme
- 85 matthew
- 86 access
- 87 yankees
- 88 987654321
- 89 dallas
- 90 austin
- 91 thunder
- 92 taylor
- 93 matrix
- 94 mobilemail
- 95 mom
- 96 monitor
- 97 monitoring
- 98 montana
- 99 moon
- 100 moscow

Load from file...

Now you see 3 types of response:

Number 1: Since the username is correct, the account will be locked on 4th attempt, so for the 1st 3 attempts we got: “Invalid Username or Password”.

Number 2: As after 3 wrong attempts, the account will be locked so after that we will get all the responses with “You have made too many incorrect login attempts.” which will have a slightly bigger response length than Number 1.

Number 3: Since after so many wrong attempts, the access to the account is locked, so even if the correct password is entered, we will not be logged in because the timer is still running. But as I said before, for correct credentials we get no response. So this response will be slightly shorter in length than the previous ones.

The screenshot shows the Burp Suite interface. At the top, there's a table of HTTP requests. The 4th request, with payload 'qwerty', is highlighted with a red box. Below the table, the 'Request' tab is selected, showing the raw HTTP request details. To the right, the 'Response' tab is selected, showing the response body. The response body contains a 'Login' form with a red box around the 'Username' input field. The response status is 3374 bytes and 658ms.

ID	Payload 1	Status	Length	Round-trip Time (ms)
4	qwerty	200	3374	658
1	123456	200	3452	758
3	12345678	200	3452	757
7	1234	200	3452	749
2	password	200	3504	719
5	123456789	200	3504	655
6	12345	200	3504	766
8	111111	200	3504	734
9	1234567	200	3504	736
10	dragon	200	3504	724
11	123123	200	3504	736

100 requests

Request

```
1 POST /login HTTP/1.1
2 Host: 0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net
3 Connection: close
4 Content-Length: 28
5 Cache-Control: max-age=0
6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
7 sec-ch-ua-mobile: ?0
8 sec-ch-ua-platform: "Windows"
9 Upgrade-Insecure-Requests: 1
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
12 Origin: https://0a7e00400433709e81e0a7ee00db00a0.web-security-academy.net
```

Response

WebSecurity Academy Username enumeration via account lock

LAB Not solved

Home | My account

Login

Username

Password

3374 bytes | 658ms

We can see the shortest length of all 3 type of responses is for **qwerty**, and since these are the correct credentials we got no response but since the timer is still running so neither we got logged in as well but to log in we have to wait for the account lock timer to expire and try the correct credentials again after 1 minute.

Step 7:

Now go to My Account after the timer resets and enter the correct credentials.

Login

Username

app

Password

.....

Log in



Congratulations, you solved the lab!

My Account

Your username is: app

Your email is: app@normal-user.net

Email

Update email

<https://www.linkedin.com/in/haroonshahid76/>

<https://github.com/haroon505>

=====